UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

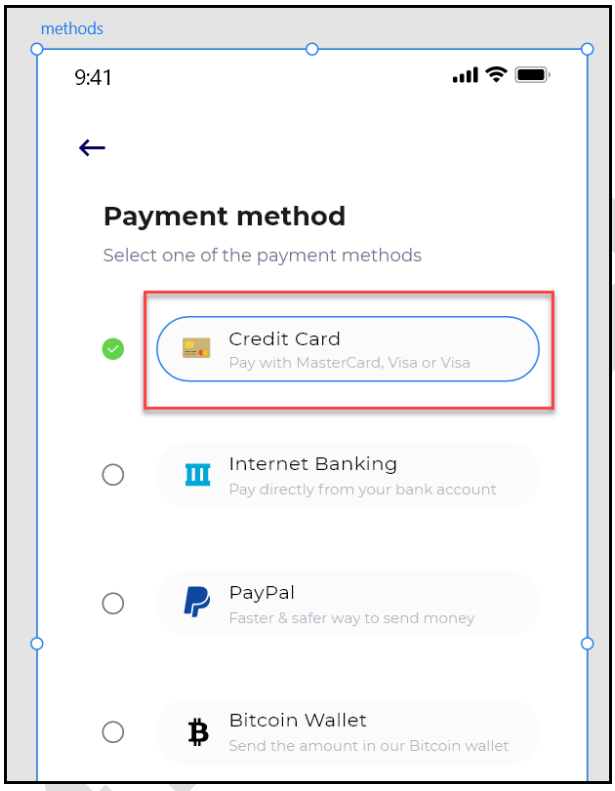
16/10/2026

Ejercicio # 1

En esta práctica vamos a continuar con el diseño de la primera pantalla para la aplicación en Flutter.

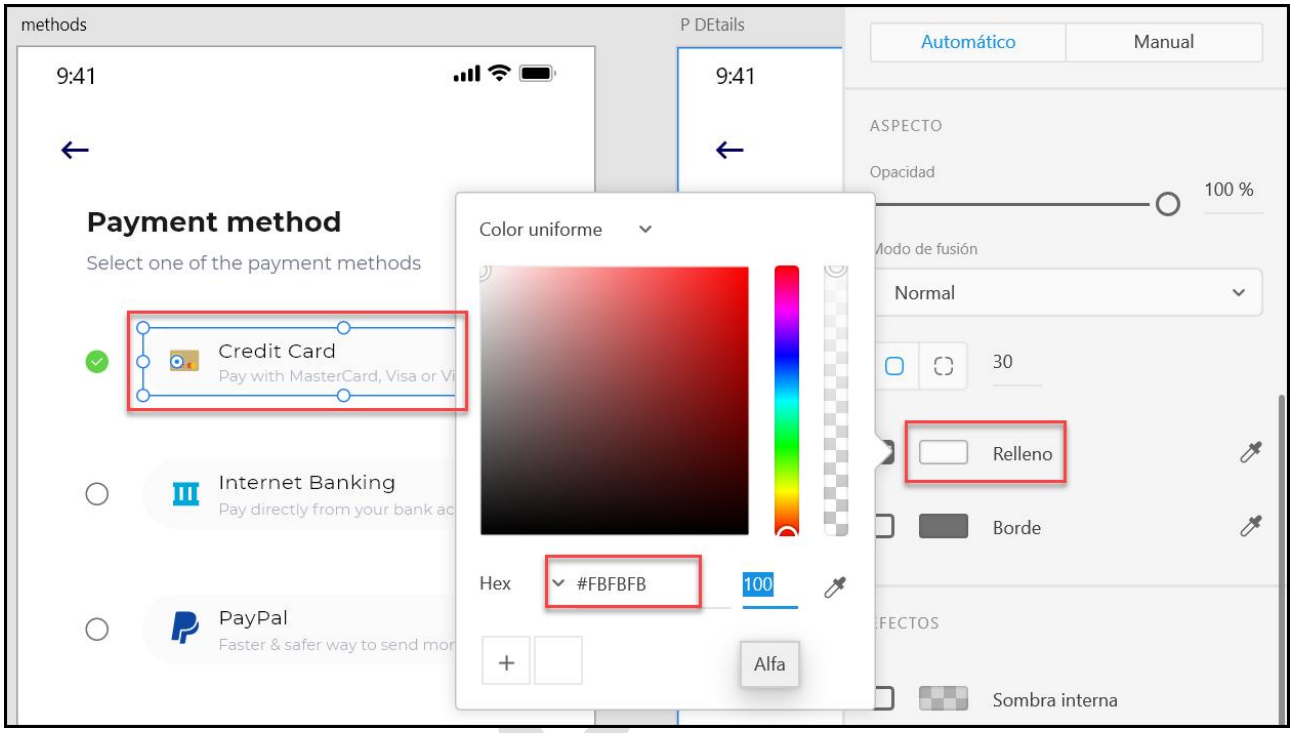
Métodos de pago #1

1. Para continuar con el diseñó **abra** en Adobe XD el **mockup** **PaymentFlow**.



2. Con la ayuda de **Adobe XD** podemos visualizar en la figura anterior que, para crear la **UI** resaltada en rojo en la figura, necesitamos un **Row**, además de este tipo de widget, para poder darle el **efecto ovalado** que tiene la fila, se requiere por ejemplo un **Container**. Note además que el **Row** está dentro de un **Container**, este es necesario para poder establecer por ejemplo los **espacios entre los métodos de pago**.

3. En **Adobe XD** se puede saber el valor exacto en hexadecimal del **color del contenedor** (en la figura anterior se muestra dentro de un rectángulo de **color rojo**, con efecto ovalado). A simple vista este parece ser de color blanco, pero en realidad el color exacto es **#FBFBFB**. Esto se muestra a continuación.



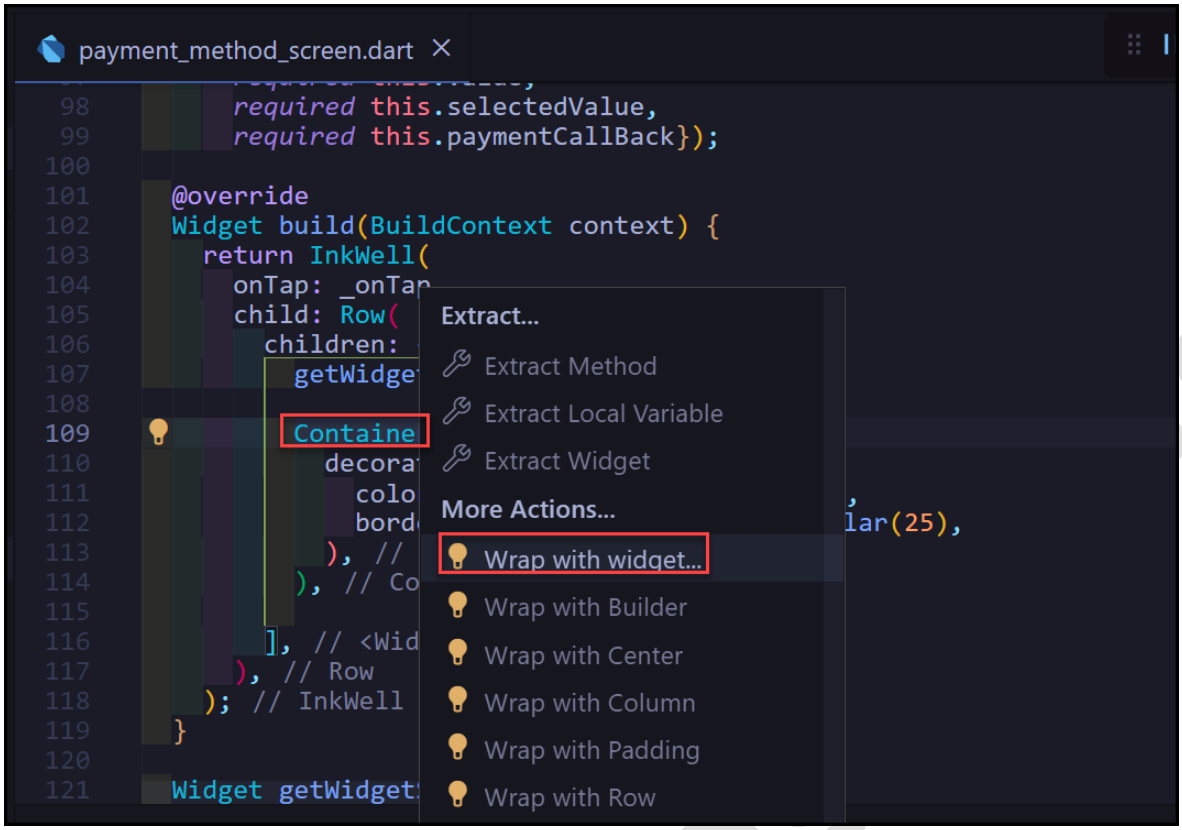


4. Una vez conocido el color del **contenedor**, en Visual Studio Code, vaya al archivo **payment_method_screen.dart**, luego a la clase **PaymentMethodWidget**, donde se está **creando** la **UI** para el **método de pago**, Desplácese dentro del método **build**, luego de la llamada al método **getWidgetState** digite el **código** siguiente, para crear en primer lugar un **Container** con los bordes **redondeados** (circular).

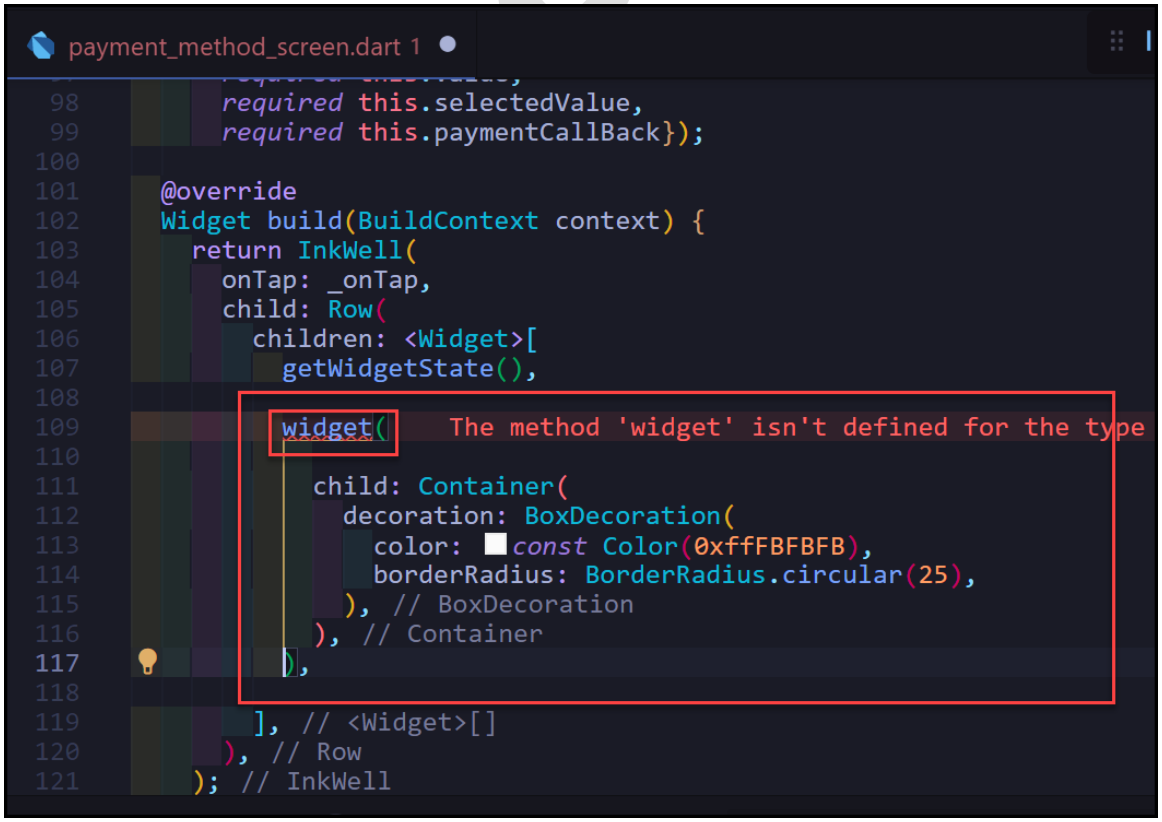


5. Si **ejecuta** el proyecto no se va a poder visualizar el **contenedor** porque el **color** es **casi blanco**, pero esa no es la razón real por la cual **no se visualiza**, incluso se puede poner **otro color de fondo** y el **container** no se va a mostrar, esto es porque el widget no tiene un **ancho y un alto**, pero para el ejercicio no se va a establecer un **ancho** y un **alto** porque es necesario que el **Container** cubra todo el espacio que deja libre el widget que está a la izquierda (este widget representa el botón de radio), por lo tanto se va a **expandir** el **Row** automáticamente, por lo que no hace falta que tenga **un ancho y un alto**, esto se aprecia mucho mejor al completar el código y ejecutar el proyecto.

6. Por lo tanto, ponga el **cursor** en el widget **Container** creado en el punto 4, con la ayuda de **Visual Studio Code** presione las teclas **CONTROL + PUNTO** (CONTROL + .), en la ventana emergente que se muestra **seleccione** la opción **Wrap with widget**.



7. Esto **genera** el siguiente código.



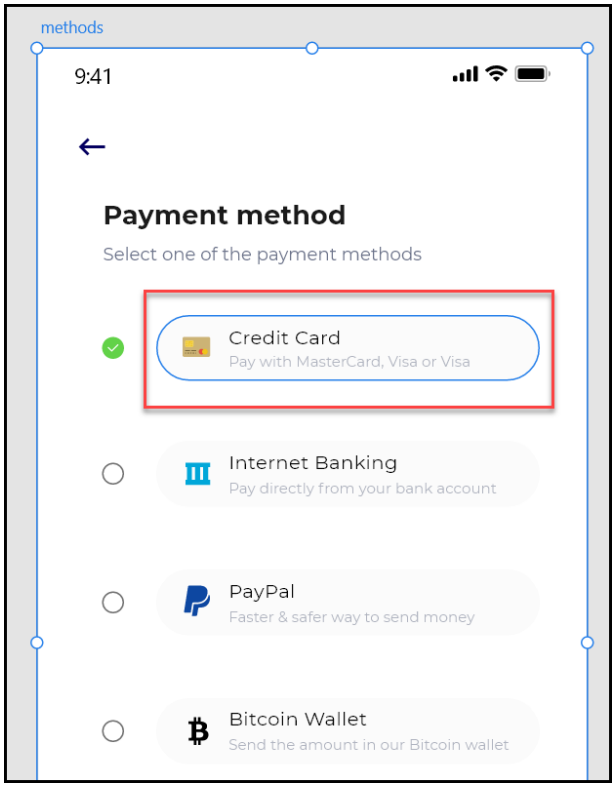
8. **Cambie** la palabra **widget** generada anteriormente por la palabra **Expanded** para crear un widget de este tipo, esto hace que el Container ahora se encuentre dentro del widget **Expanded**. El **código** se muestra a continuación.

payment_method_screen.dart

```
98        required this.selectedValue,
99        required this.paymentCallBack});
100
101   @override
102   Widget build(BuildContext context) {
103     return InkWell(
104       onTap: _onTap,
105       child: Row(
106         children: <Widget>[
107           getWidgetState(),
108
109           Expanded (
110
111             child: Container(
112               decoration: BoxDecoration(
113                 color: const Color(0xffFBFBFB),
114                 borderRadius: BorderRadius.circular(25),
115               ), // BoxDecoration
116             ), // Container
117           ), // Expanded
118
119         ], // <Widget>[]
120       ), // Row
121     ); // InkWell
```

- Los widgets **no tienen la capacidad de expandirse**, a excepción de en ciertos escenarios. Sin embargo, si queremos que un widget se **expanda** tendremos que indicárselo de manera explícita. Para esto se utiliza en conjunto widgets como el **Column** y **Row** con el widget **Expanded** para que las filas y columnas se expandan automáticamente según el contenido de los widgets.
- Se puede establecer el **alto** para un widget **Expanded**, sin embargo, como ya se dijo anteriormente, para el ejemplo el **alto se va a establecer automáticamente** de acuerdo con los controles que estén dentro del **Expanded**.

9. **Abra** nuevamente el **mockup** para tener claro lo que estamos **realizando**.



- De acuerdo con el **mockup**, hasta el momento todo lo que está en la figura anterior en **color rojo** (la línea en rojo se puso en la figura anterior solo como referencia) lo vamos a colocar dentro de un **Container**, el Container está dentro de un **Expanded** para que el ancho de lo que está dentro del **Expanded** tenga el ancho de acuerdo con los widgets que están dentro de este (todo lo que está dentro del rectángulo en color en rojo en la figura anterior).
- Vamos a **agregar** los **widgets** necesarios para tener el resultado que se muestra en la **figura** anterior, por ahora tenemos el **Container** que está **dentro** del **Expanded**.

10. En la clase **PaymentMethodWidget**, método **build**, al final del **Container**, cree por ahora **una fila** (Row) con un **ancho** específico, para esto se utiliza la propiedad **child** del **Container**, el código se muestra en la siguiente figura.

Flutter-Clay Aldás

payment_method_screen.dart

```dart
Widget build(BuildContext context) {
  return InkWell(
    onTap: _onTap,
    child: Row(
      children: <Widget>[
        getWidgetState(),

        Expanded(
          child: Container(
            decoration: BoxDecoration(
              color: const Color(0xffFBFBFB),
              borderRadius: BorderRadius.circular(25),
            ), // BoxDecoration

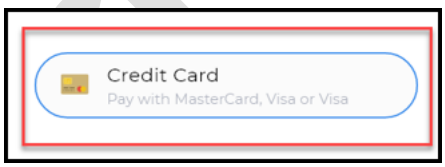
            child: Row(    Use 'const' with the constructor to improve performance
              mainAxisSize: MainAxisSize.min,
            ), // Row

          ), // Container
        ), // Expanded

      ], // <Widget>[]
    ), // Row
  ); // InkWell
```

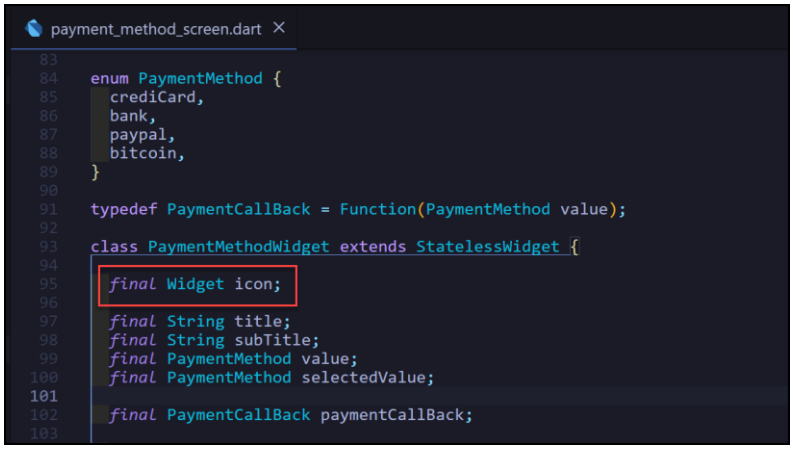
11. Es necesario recordar que las filas tienen un **ancho infinito**, por lo que, si se ubica una **fila y dentro otra fila** sin que tengan un **ancho** explicito, ambas filas van a buscar tener **todo el espacio disponible** en el dispositivo, por lo que se va a generar un **error**, en este caso para el ejemplo anterior el widget **Row** más interno se ha **establecido** con un **ancho mínimo** (en el código anterior: mainAxisSize: **MainAxisSize.min**).

12. Siguiendo el **mockup**, ahora se vamos a agregar dentro del **Row** en el parámetro **child** un **icono** y una **columna** para los textos. Dentro de la **columna** tendríamos el **título** y el **subtítulo** que se muestra para cada tipo de pago.



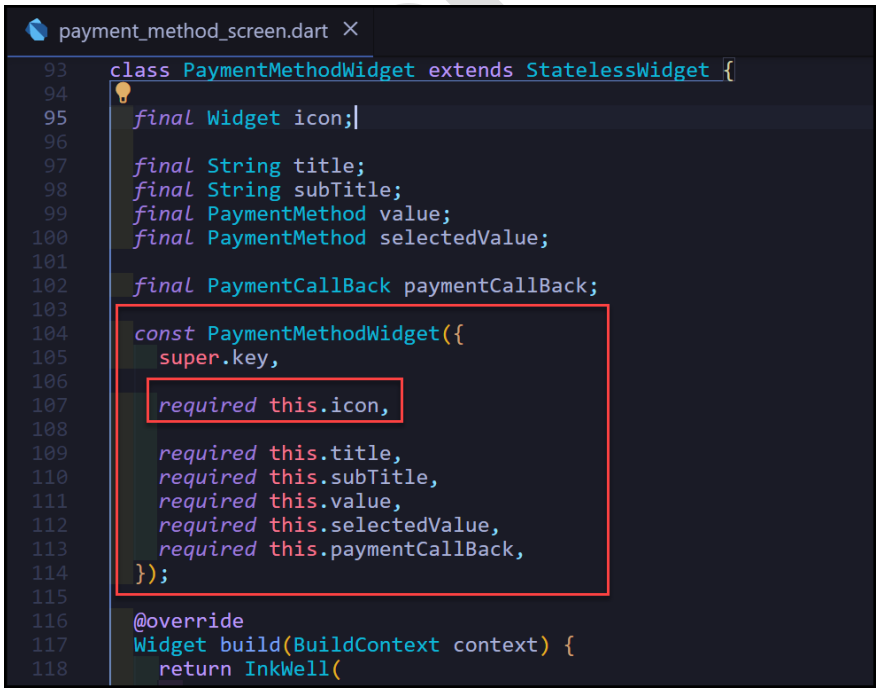
13. Para establecer el **icono** del tipo de pago, lo primero que debemos hacer es **agregar** una variable a la clase que representa el **método de pago**, es decir lo declaramos en la clase **PaymentMethodWidget** y le ponemos a la variable el nombre **icon**, esta

variable debe ser de tipo **Widget** para acepte cualquier **componente** (Widget). Esto se muestra en la siguiente figura.



14. Ahora es necesario **agregar** la variable creada **como parámetro** al **constructor** de la clase, para tener un **mejor orden** lo vamos a agregar como **segundo parámetro** (luego del parámetro **key**), aunque si nos **ayudamos** para esto de **Visual Studio Code** este lo va al poner **al final** de todos los parámetros, por lo tanto, lo **vamos a realizar manualmente**. El código del **constructor** agregado el parámetro llamado **icon** de forma manual (que es la variable **icon de la clase**) es el siguiente.

Constructor modificado.



15. Ahora **vaya** a la clase **PaymentMethodState** y dentro del método **build** **modifique** las **dos llamadas** al constructor de la clase **PaymentMethodWidget** para que **utilice** el nuevo parámetro (**icon**). Para los dos **iconos** vamos a utilizar el paquete de **Awesome** con un **color**, aunque se puede utilizar las imágenes importadas desde Adobe XD y Figma.

Constructor modificado.

payment_method_screen.dart

```
49          PaymentMethodWidget(
50
51            icon: const Icon(
52              FontAwesomeIcons.creditCard,
53              color: Colors.orange,
54            ), // Icon
55
56            title: 'Tarjeta de Crédito',
57            subTitle: 'Pague con Mastercard o Visa',
58            value: PaymentMethod.paypal,
59            selectedValue: paymentSelected,
60            paymentCallBack: (value) {
61              setState(() {
62                paymentSelected = value;
63              });
64            },
65          ), // PaymentMethodWidget
66          PaymentMethodWidget(
67
68            icon: const Icon(
69              FontAwesomeIcons.warehouse,
70              color: Colors.orange,
71            ), // Icon
72
73            title: 'Transferencia bancaria',
74            subTitle: 'Pague directo desde su cuenta',
75            value: PaymentMethod.bank,
76            selectedValue: paymentSelected,
77            paymentCallBack: (value) {
78              setState(() {});
79              paymentSelected = value;
80            },
```

16. En el método **build** de la clase **PaymentMethodWidget** continue con la **edición** del **Row**. Dentro del **Row** agregue un **icono** para el **método de pago** y luego un **Column** y dentro de este widget ubicamos 2 **Text**, para el **título** y **subtítulo** respectivamente. Para esto utilice la propiedad **children** del widget **Row**.

payment_method_screen.dart

```dart
@override
Widget build(BuildContext context) {
  return InkWell(
    onTap: _onTap,
    child: Row(
      children: <Widget>[
        getWidgetState(),
        Expanded(
          child: Container(
            decoration: BoxDecoration(
              color: const Color(0xffFBFBFB),
              borderRadius: BorderRadius.circular(25),
            ), // BoxDecoration

            child: Row(
              mainAxisSize: MainAxisSize.min,

              children: <Widget> [
                icon,
                Column(
                  children: <Widget> [
                  Text(title),
                  Text(subTitle),
                  ], // <Widget>[]
                ) // Column
              ], // <Widget>[]
            ), // Row

          ), // Container
        ), // Expanded
      ], // <Widget>[]
    ), // Row
```

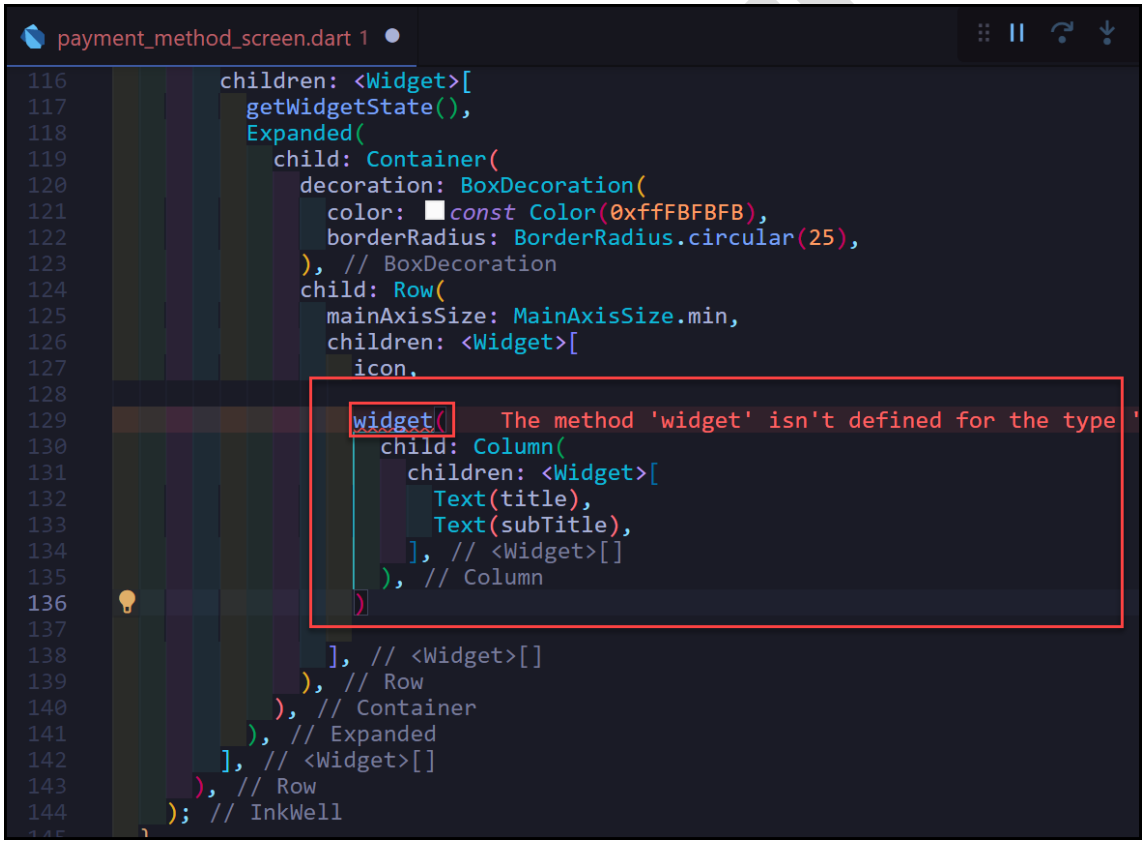
17. Si se **ejecuta** el proyecto tendremos la siguiente salida.



18. Vamos a **expandir** el **Column** para mejorar la UI. Para esto ponga el **cursor** en el widget **Column** creado anteriormente y con la ayuda de **Visual Studio Code** presione las teclas **CONTROL + PUNTO** (CONTROL + .), en la ventana emergente que se muestra **seleccione** la opción **Wrap with widget**.



19. Esto **genera** el siguiente código.

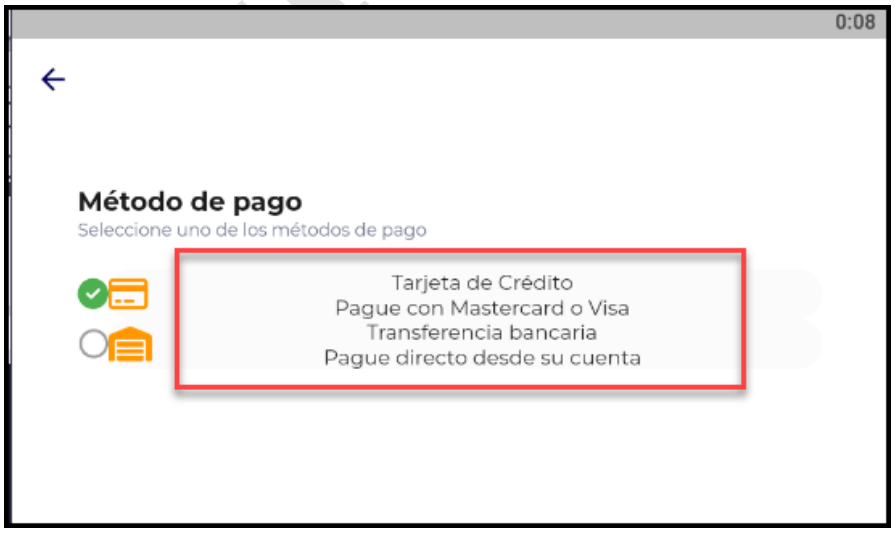


20. **Cambie** la palabra **widget** generada anteriormente por la palabra **Expanded**. El **código** resultante se muestra a continuación.

payment_method_screen.dart

```
116          children: <Widget>[
117            getWidgetState(),
118            Expanded(
119              child: Container(
120                decoration: BoxDecoration(
121                  color: const Color(0xffFBFBFB),
122                  borderRadius: BorderRadius.circular(25),
123                ), // BoxDecoration
124                child: Row(
125                  mainAxisSize: MainAxisSize.min,
126                  children: <Widget>[
127                    icon,
128
129                    Expanded(
130                      child: Column(
131                        children: <Widget>[
132                          Text(title),
133                          Text(subTitle),
134                        ], // <Widget>[]
135                      ), // Column
136                    ) // Expanded
137
138                  ], // <Widget>[]
139                ), // Row
140              ), // Container
141            ), // Expanded
142          ], // <Widget>[]
143        ), // Row
144      ); // InkWell
145    }
```

21. Al **ejecutar** el proyecto tenemos la siguiente pantalla. La columna ahora ocupa todo el espacio disponible restante.

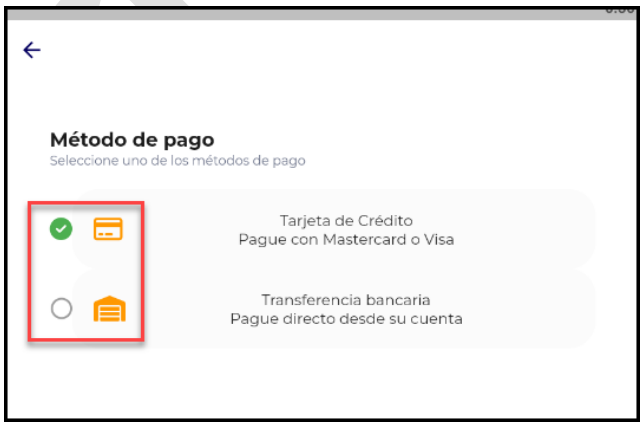


22. En la figura anterior los **iconos** están muy pegados uno del otro, el **icono** que está a la **derecha** está dentro del **Container**, por lo tanto, agregue un **Padding** para establecer un espaciado, este es de 20 pixeles en el ejemplo para todo el contorno (esto es **padding**: **const EdgeInsets.all(20)).**

payment_method_screen.dart

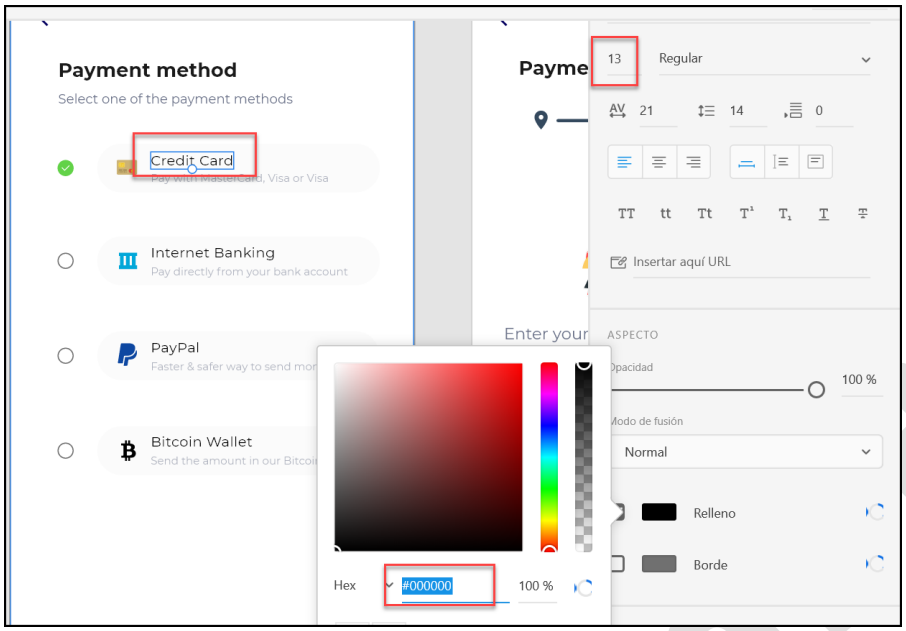
```
111   @override
112   Widget build(BuildContext context) {
113     return InkWell(
114       onTap: _onTap,
115       child: Row(
116         children: <Widget>[
117           getWidgetState(),
118           Expanded(
119
120             child: Container(
121
122               padding: const EdgeInsets.all(20),
123
124               decoration: BoxDecoration(
125                 color: const Color(0xffFBFBFB),
126                 borderRadius: BorderRadius.circular(25),
127               ), // BoxDecoration
128               child: Row(
129                 mainAxisSize: MainAxisSize.min,
130                 children: <Widget>[
131                   icon,
132                   Expanded(
133                     child: Column(
134                       children: <Widget>[
135                         Text(title),
136                         Text(subTitle),
137                       ], // <Widget>[]
138                     ), // Column
139                   ) // Expanded
140                 ], // <Widget>[]
141               ), // Row
142             ), // Container
```

23. Al **ejecutar** el proyecto se puede visualizar que ahora hay más espacio entre los **iconos**.



24. Utilice **Adobe XD** para conocer el **tamaño** y **color** de la **fuente** para el **título** y **subtítulo** usada en el **mockup**.

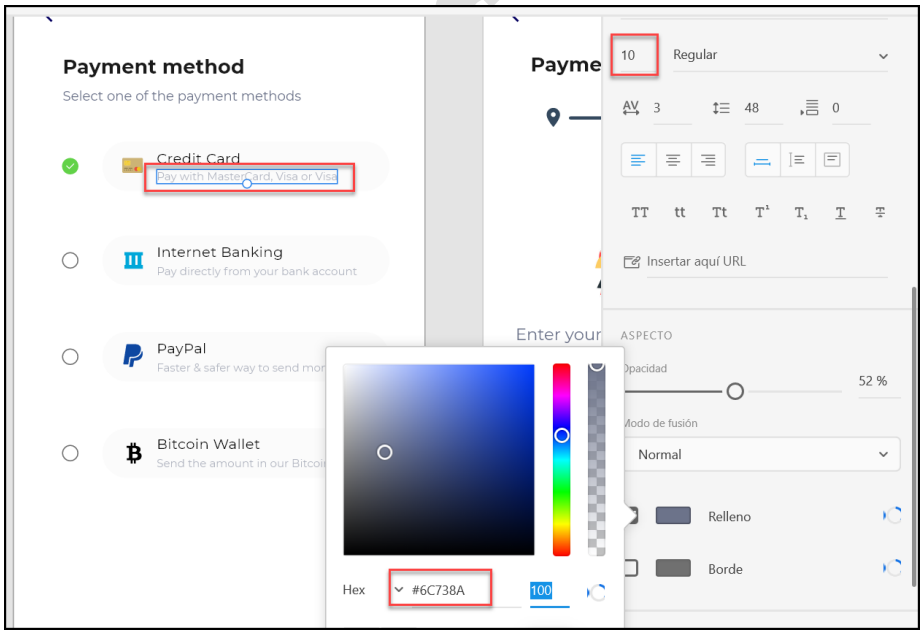
Texto para el título seleccionado, muestra el tamaño y el color de este.



Tamaño: 13

Color: #000000

Texto para el subtítulo seleccionado, muestra el tamaño y el color de este.



Tamaño: 10

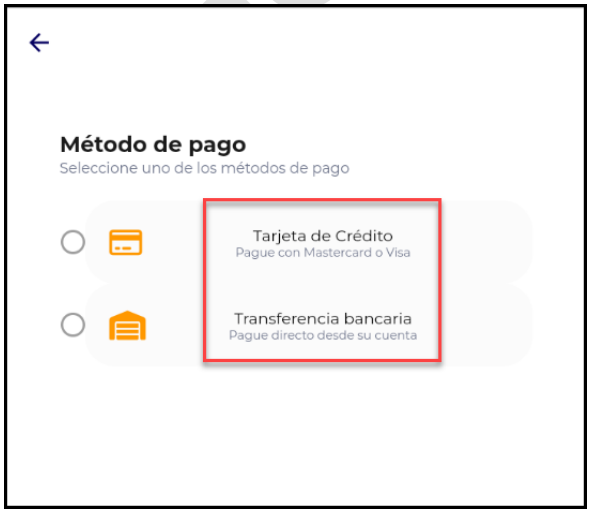
Color: #6C738A

25. Con estos datos conocidos, en el proyecto **establezca** estos valores para el tamaño y el color en el **título** y **subtítulo** en los **Text**. Vaya a la clase **PaymentMethodWidget** y dentro del **build** en **Column** donde están lo **Text**, digite el código requerido.

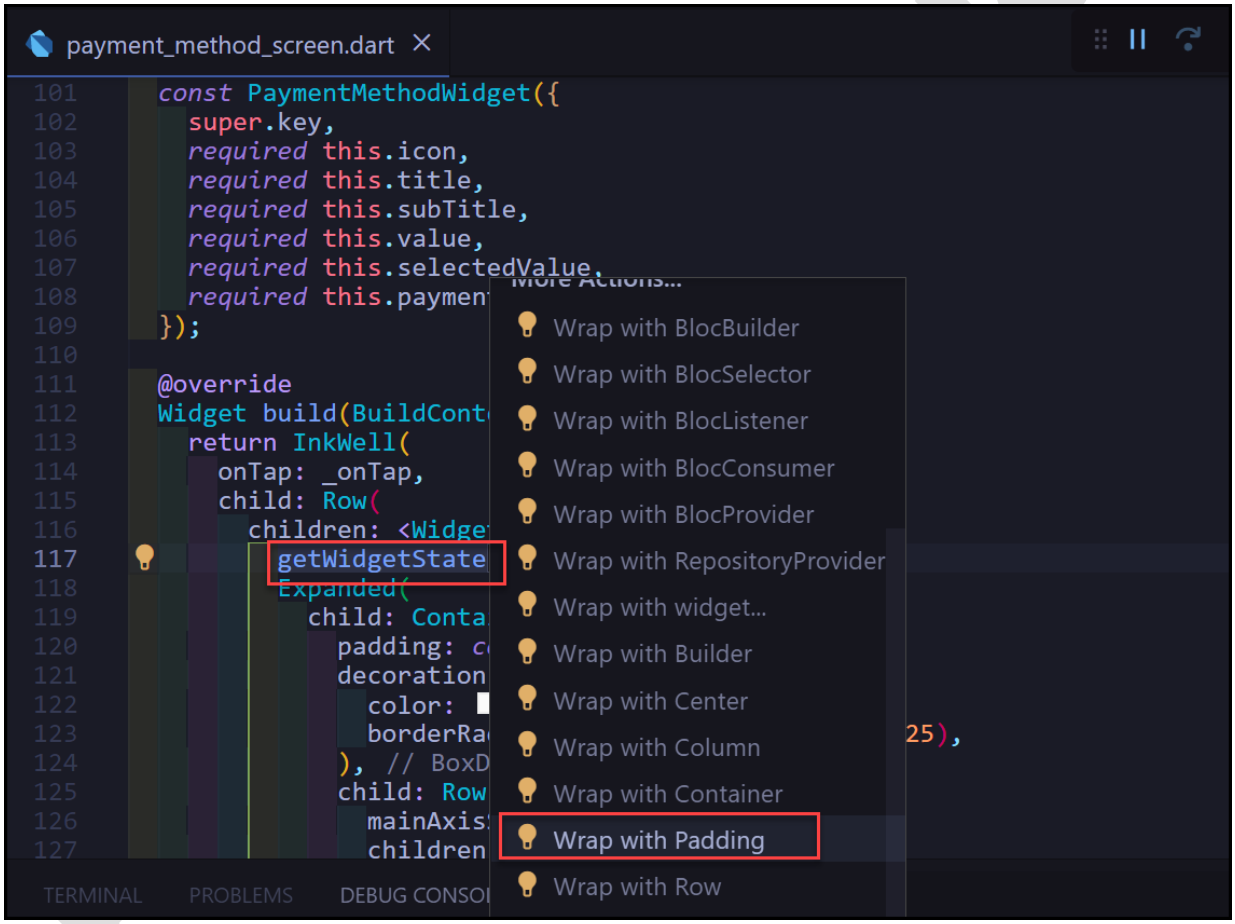
payment_method_screen.dart

```
126                mainAxisSize: MainAxisSize.min,
127                children: <Widget>[
128                  icon,
129                  Expanded(
130                    child: Column(
131                      children: <Widget>[
132
133                        Text(
134                          title,
135                          style: const TextStyle(
136                            color: Color(0xff000000),
137                            fontSize: 13,
138                          ), // TextStyle
139                        ), // Text
140
141                        Text(
142                          subTitle,
143                          style: const TextStyle(
144                            color: Color(0xff6C738A),
145                            fontSize: 10,
146                          ), // TextStyle
147                        ), // Text
148
149                      ], // <Widget>[]
150                    ), // Column
151                  ) // Expanded
```

26. **Ejecute** el proyecto.



27. Hace falta agregar más **espaciado horizontal** en el primer widget, es decir en el botón de **Radio** en la figura anterior, **esta parte se creó** con el método llamado **getWidgetState**, por lo tanto, como este método **regresa** un **widget**, entonces vamos a utilizar un **widget** de tipo **Padding** para **envolverlo** en este **widget** y **agregar** más **espaciado**. Para esto vaya a la clase **PaymentMethodWidget**, dentro del método **build** ponga el mouse en el método **getWidgetState** (luego de la propiedad **children**) presione las teclas **CLIC + PUNTO** (CLIC + .), de la ventana emergente **seleccione** la opción **Wrap with Padding** para envolver el código seleccionado dentro de un widget Padding.

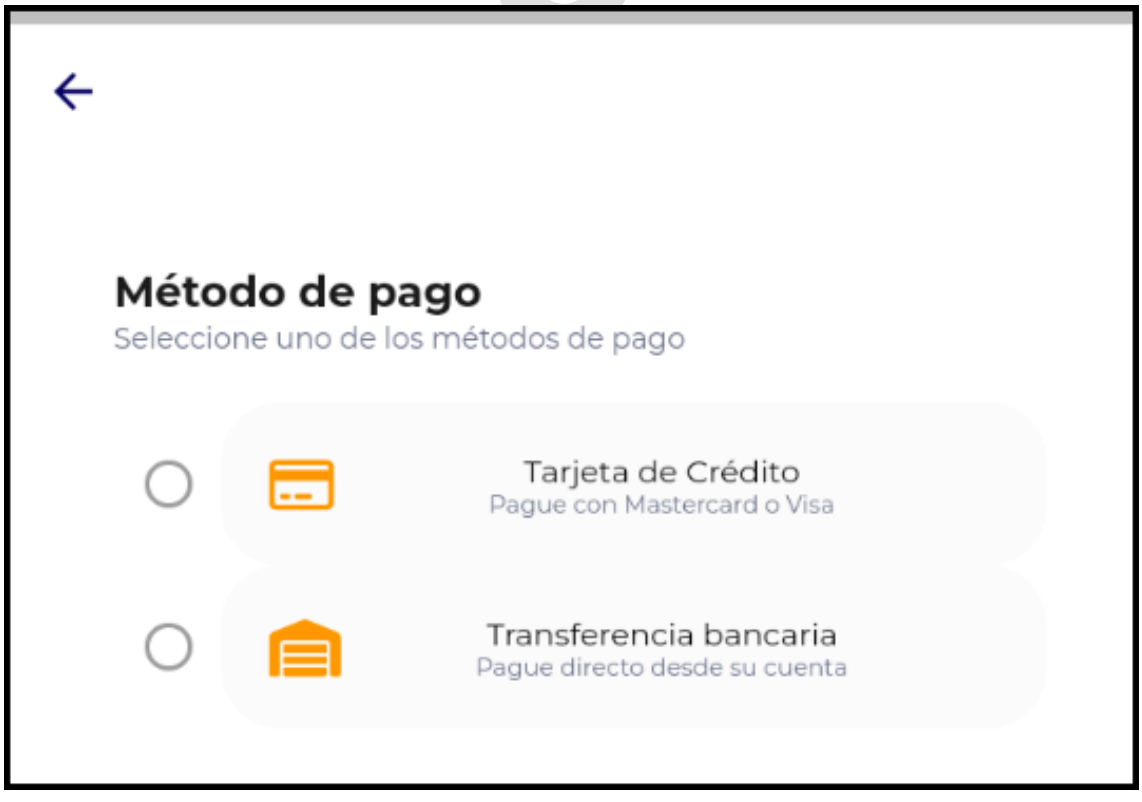


28. **Dentro** del código generado por **Visual Studio Code** agregue en la propiedad **padding** del **Padding** relleno a la **izquierda** y a la **derecha**, para esto digite el siguiente código.

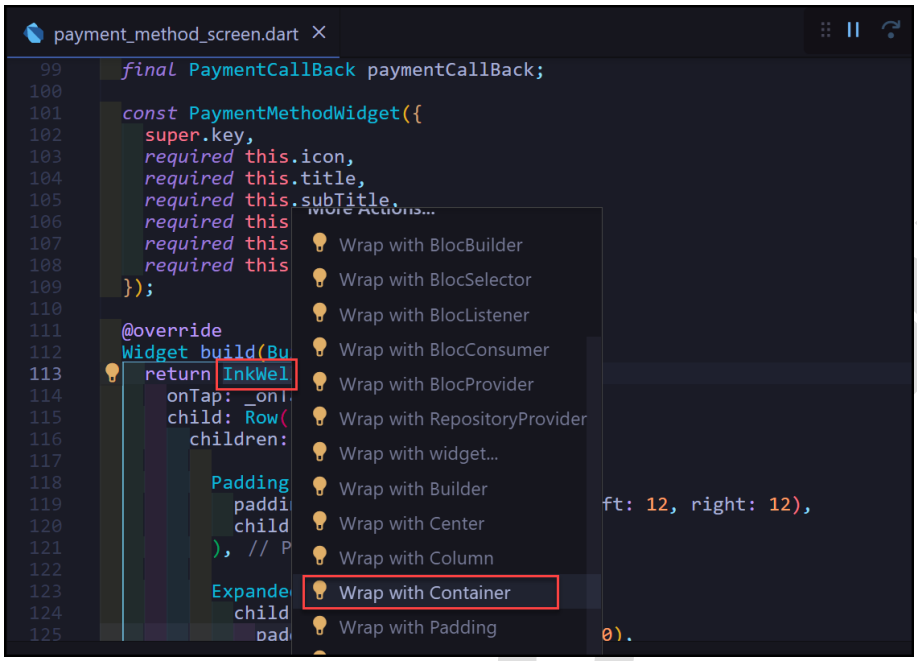
payment_method_screen.dart

```
104      required this.title,
105      required this.subTitle,
106      required this.value,
107      required this.selectedValue,
108      required this.paymentCallBack,
109    });
110
111    @override
112    Widget build(BuildContext context) {
113      return InkWell(
114        onTap: _onTap,
115        child: Row(
116          children: <Widget>[
117
118            Padding(
119              padding: const EdgeInsets.only(left: 12, right: 12),
120              child: getWidgetState(),
121            ), // Padding
122
123            Expanded(
124              child: Container(
125                padding: const EdgeInsets.all(20),
126                decoration: BoxDecoration(
127                  color: const Color(0xffFBFBFB),
128                  borderRadius: BorderRadius.circular(25),
129                ), // BoxDecoration
130                child: Row(
```

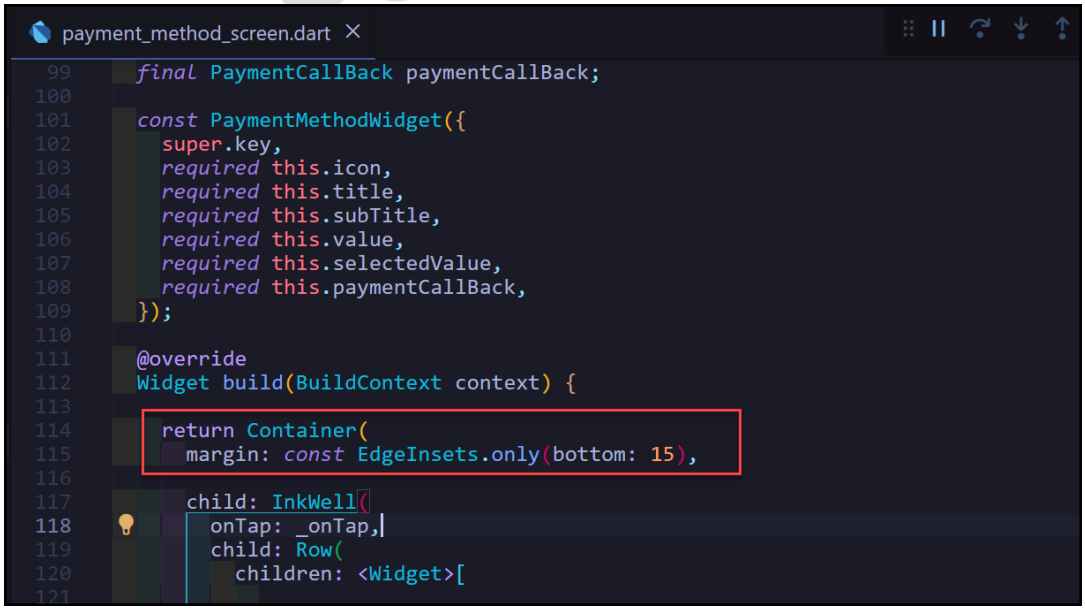
29. **Ejecute** el proyecto.



30. Continuamos, si revisamos el **mockup** puede notar que **hay más espaciado vertical** entre cada **método de pago**. Para agregar más espaciado vertical vaya a la clase **PaymentMethodWidget**, dentro el método **build** ponga el **mouse** sobre el widget **InkWell**. presione las teclas **CLIC + PUNTO** (CLIC + .), de la ventana emergente **seleccione** la opción **Wrap with Container**.

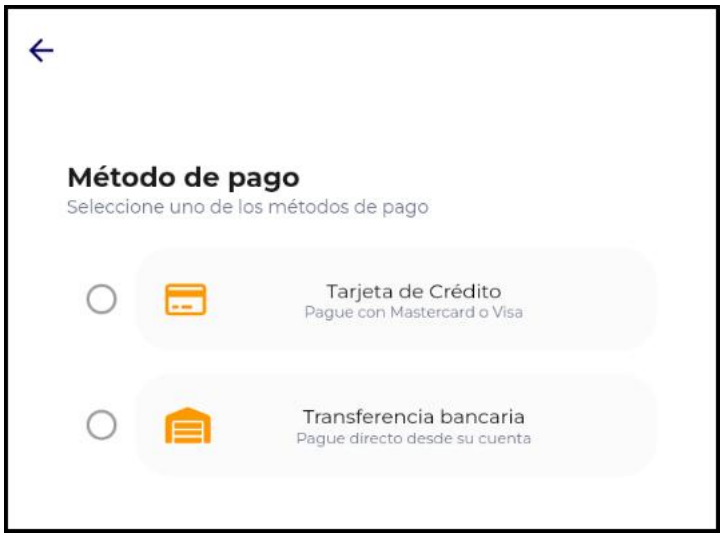


31. Use la propiedad **margin** para agregar margen en la parte de abajo (**bottom**) del Container, el código se muestra a continuación.

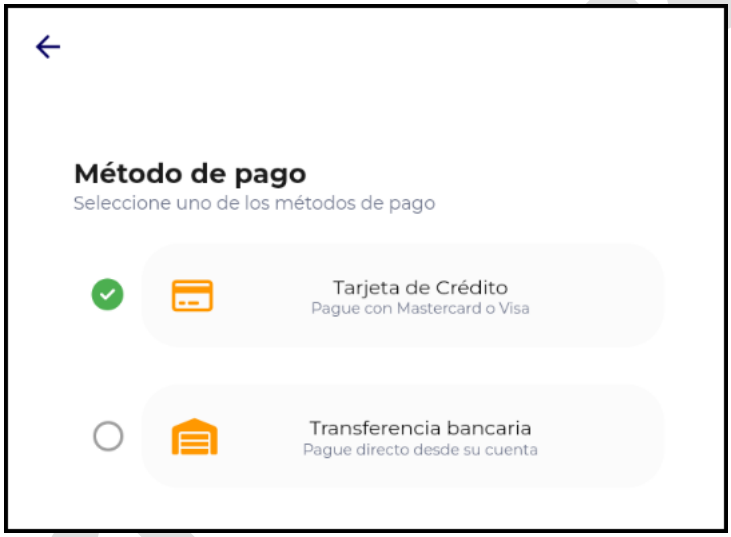


32. **Ejecute** el proyecto.

Ventana sin el método de pago seleccionado.



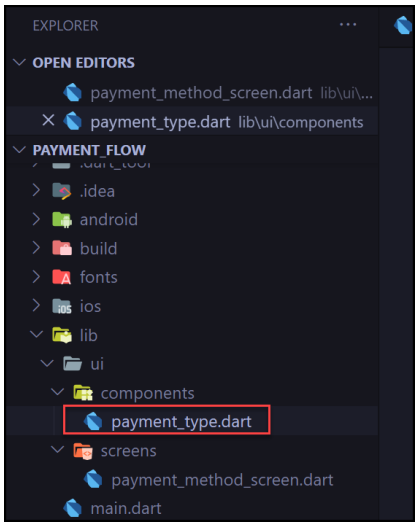
Ventana con el método de pago seleccionado.



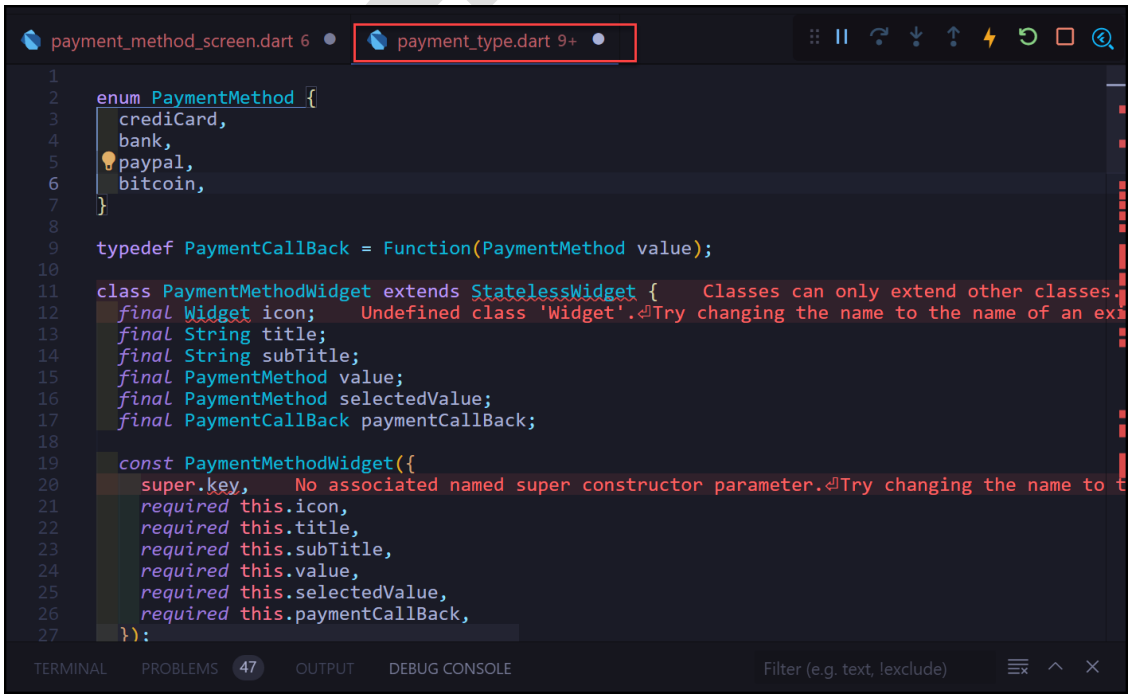
33. Si analiza el código creado hasta el momento, puede notar que hay código **repetitivo**, lo mejor es **separarlo** en otros **archivos** y **carpetas** para facilidad de mantenimiento y lectura del código. Por lo tanto, antes de crear la **UI** para los siguientes métodos de pago para **PayPal** y **Bitcon**, vamos a ordenar mejor el código.

34. Para esto, **dentro** de la carpeta **components** del proyecto **cree** el archivo llamado **payment_type.dart**.

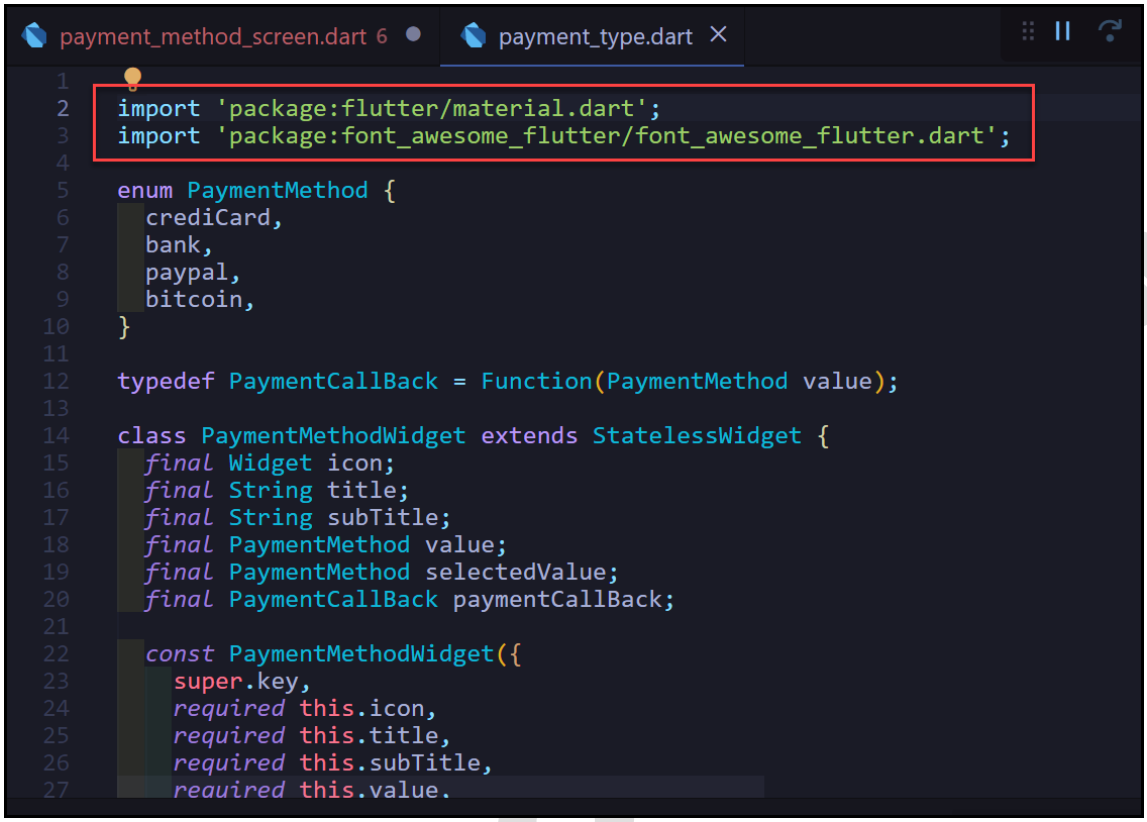
Archivo creado.



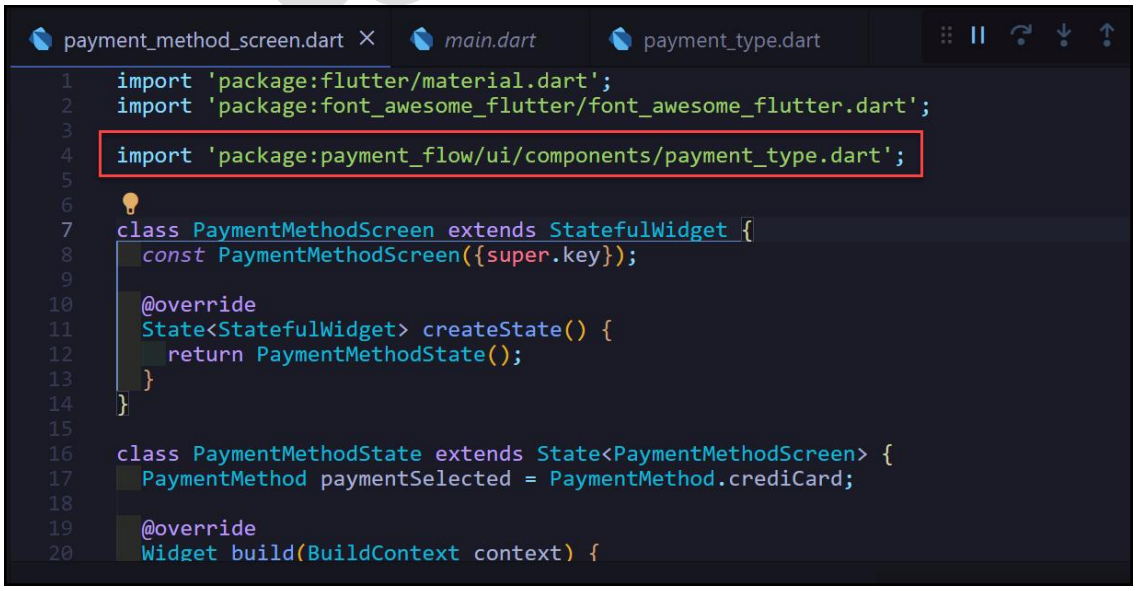
35. Ahora desde el archivo **payment_method_screen.dart** **pase el código** desde la **enumeración** llamada **PaymentMethod** hasta el **final** de este archivo. Es decir, pasamos la enumeración **PaymentMethod**, el tipo de dato **PaymentCallBack** y la clase **PaymentMethodWidget** al archivo **payment_type.dart**. Esto se muestra en la figura siguiente.



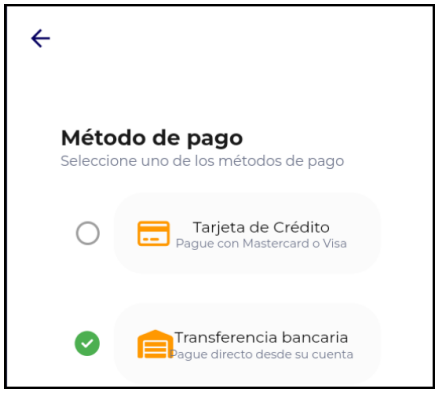
36. En el **inicio** del archivo **payment_type.dart** importe los **paquetes** para **material** y **font_awesome_flutter**.



37. Ahora al **inicio** del archivo **payment_method_screen.dart** importe el nuevo archivo creado llamado **payment_type.dart**.



38. Si se **ejecuta** el proyecto todo debería funcionar **correctamente**.



Nota	• Si copiar el código al nuevo archivo **payment_type.dart** se genera un error, **detenga completamente la aplicación** y vuelva a compilar el proyecto.

39. Vamos a **crear** ahora la UI para los métodos de pago de **Paypal** y **Bitcoin**, para esto vaya al **archivo** **payment_method_screen.dart**, luego desplácese **dentro** del método **build** de la clase **_PaymentMethodState**, al **final** de este método (**build**), **señale** todo el código de la última llamada al método **PaymentMethodWidget** en el cual se hace la UI para el método de pago por **Transferencia bancaria**, esto se indica en la siguiente figura.

Código **señalado** método de pago por **Transferencia bancaria** para copiarlo.



40. **Copie** el código marcado anteriormente y **péguelo** dos veces a continuación, ahora tendremos 4 secciones para la **UI** de los **métodos de pago**, esto se muestra en la figura siguiente.

payment_method_screen.dart

```
                color: Color(0xff6C738A),
              ), // TextStyle
            ), // Text
            const SizedBox(
              height: 20,
            ), // SizedBox

            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.creditCard,
                color: Colors.orange,
              ), // Icon
              title: 'Tarjeta de Crédito',
              subTitle: 'Pague con Mastercard o Visa',
              value: PaymentMethod.paypal,
              selectedValue: paymentSelected,
              paymentCallBack: (value) {
                setState(() {
                  paymentSelected = value;
                });
              },
            ), // PaymentMethodWidget

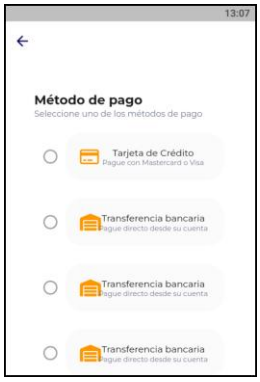
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.warehouse,
                color: Colors.orange,
              ), // Icon
              title: 'Transferencia bancaria',
              subTitle: 'Pague directo desde su cuenta',
              value: PaymentMethod.bank,
              selectedValue: paymentSelected,
              paymentCallBack: (value) {
                setState(() {});
                paymentSelected = value;
              },
            ), // PaymentMethodWidget

            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.warehouse,
                color: Colors.orange,
              ), // Icon
              title: 'Transferencia bancaria',
              subTitle: 'Pague directo desde su cuenta',
              value: PaymentMethod.bank,
              selectedValue: paymentSelected,
              paymentCallBack: (value) {
                setState(() {});
                paymentSelected = value;
              },
            ), // PaymentMethodWidget

            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.warehouse,
                color: Colors.orange,
              ), // Icon
              title: 'Transferencia bancaria',
              subTitle: 'Pague directo desde su cuenta',
              value: PaymentMethod.bank,
              selectedValue: paymentSelected,
              paymentCallBack: (value) {
                setState(() {});
                paymentSelected = value;
              },
            ), // PaymentMethodWidget

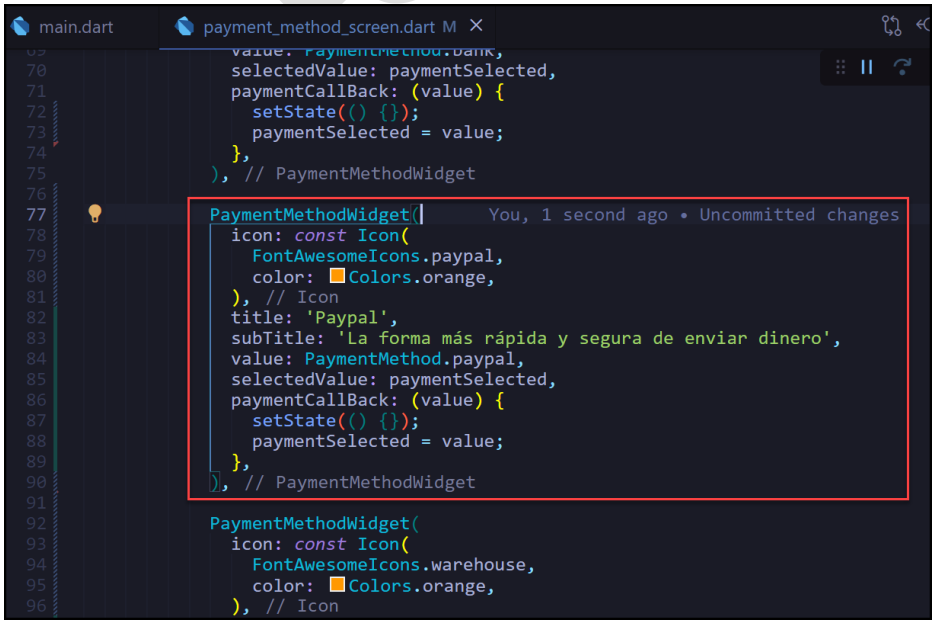

          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

41. Si se **ejecuta** proyecto tendremos el siguiente resultado.



42. Como se puede apreciar en la figura anterior, la **UI ahora tiene 4 items** para los métodos de **pago**, y en las **2 últimas** se repite la **UI** porque el código lo **copiamos** y **pegamos** de la forma de pago anterior (Esto es: **Transferencia bancaria**). Toda la lógica del método de pago lo tenemos en el widget llamado **PaymentMethodWidget** el cual se encuentra en el archivo **payment_type.dart**.

43. **Vamos** a cambiar el **texto** del 3er item de la **forma de pago**, para que muestre en la propiedad **title** "**Paypal**", en la propiedad **subtitle**: "**La forma más rápida y segura de enviar dinero**", y también cambiamos las propiedades **icon** para que muestre un icono relacionado con la forma de pago (**Paypal**), y la propiedad **value** con el valor **PaymentMethod.paypal** respectivamente. Esto se muestra a continuación.

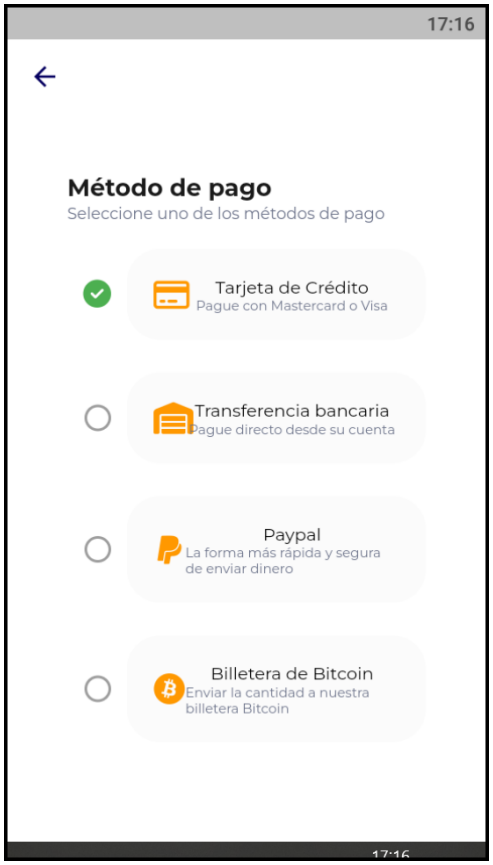


44. La **4ta forma de pago** debe mostrar en la propiedad **title** "**Billetera de Bitcoin**", para la propiedad **subtitle**: "**Enviar la cantidad a nuestra billetera Bitcoin**", cambiamos las propiedades **icon** para que muestre un icono relacionado con la forma de pago (**Bitcoin**), y la propiedad **value** con el valor **PaymentMethod.bitcoin** respectivamente. Esto se muestra a continuación.

payment_method_screen.dart

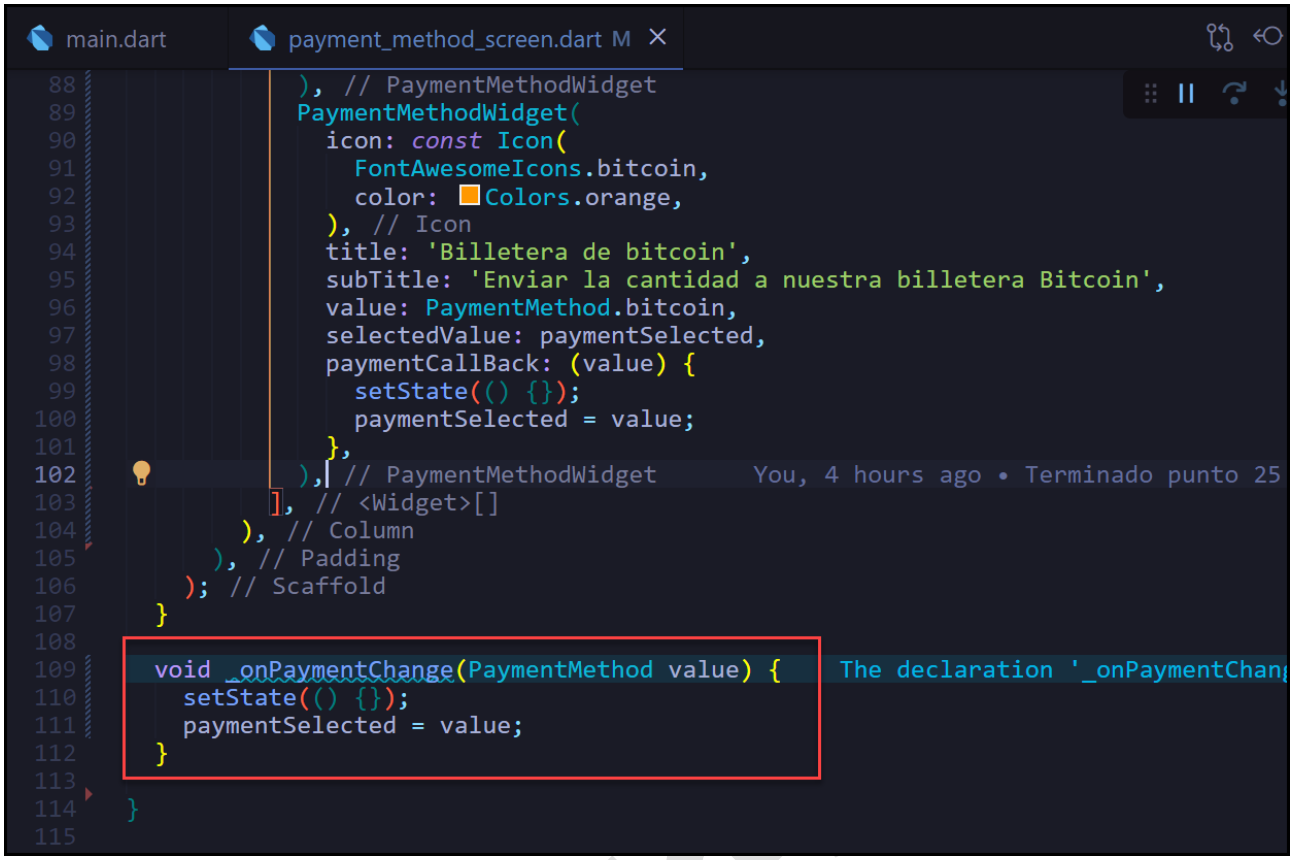
```
92                  paymentSelected = value;
93                },
94              ), // PaymentMethodWidget
95
96
97              PaymentMethodWidget(
98                icon: const Icon(
99                  FontAwesomeIcons.bitcoin,
100                 color: Colors.orange,
101               ), // Icon
102               title: 'Billetera de Bitcoin',
103               subTitle: 'Enviar la cantidad a nuestra billetera Bitcoin',
104               value: PaymentMethod.bitcoin,
105               selectedValue: paymentSelected,
106               paymentCallBack: (value) {
107                 setState(() {});
108                 paymentSelected = value;
109               },
110             ), // PaymentMethodWidget
111
112
113           ], // <Widget>[]
114         ), // Column
115       ), // Padding
116     ); // Scaffold
117   }
118 }
119
```

45. Al **ejecutar** el proyecto se debería obtener la siguiente **UI**.



46. **Puesto** que al llamar 4 veces al método **PaymentMethodWidget**, para los métodos diferentes tipos de pago, el código que se pasa para la propiedad **paymentCallBack** se **repite**, vamos a crear un **método privado** para reutilizar el código. Abra el archivo **payment_method_screen**, al final de la clase **PaymentMethodState**, luego del **Scaffold**, cree el método con el nombre **_onPaymentChange**, **dentro de este método** ponemos el **código** que se muestra en la propiedad **paymentCallBack** de cada método de pago para **cambiar el estado del método de pago**. Se puede ayudar de **Visual Studio Code** para **facilitar** este proceso de generar el método. El código para el **método** se muestra en la figura a continuación.

flutterClay Aldás



47. Ahora reemplace el código de la propiedad **paymentCallBack** que se **repite** en cada método de pago, por la llamada al método creado **_onPaymentChange**. Esto lo hacemos en las 4 formas de pago. A **continuación,** el código respectivo.

payment_method_screen.dart M

```dart
            ), // Text
            const Text(      You, 5 hours ago • Terminado punto 25 de la práctica 005
              'Seleccione uno de los métodos de pago',
              style: TextStyle(
                fontSize: 12,
                color: Color(0xff6C738A),
              ), // TextStyle
            ), // Text
            const SizedBox(
              height: 20,
            ), // SizedBox
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.creditCard,
                color: Colors.orange,
              ), // Icon
              title: 'Tarjeta de crédito',
              subTitle: 'Pague con Mastercard o Visa',
              value: PaymentMethod.creditCard,
              selectedValue: paymentSelected,

              paymentCallBack:_onPaymentChange,

            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.warehouse,
                color: Colors.orange,
              ), // Icon
              title: 'Transferencia bancaria',
              subTitle: 'Pague directo desde su cuenta',
              value: PaymentMethod.bank,
              selectedValue: paymentSelected,

              paymentCallBack: _onPaymentChange,

            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.paypal,
                color: Colors.orange,
              ), // Icon
              title: 'Paypal',
              subTitle: 'La forma más rápida y segura de enviar dinero',
              value: PaymentMethod.paypal,
              selectedValue: paymentSelected,

              paymentCallBack: _onPaymentChange,

            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.bitcoin,
                color: Colors.orange,
              ), // Icon
              title: 'Billetera de bitcoin',
              subTitle: 'Enviar la cantidad a nuestra billetera Bitcoin',
              value: PaymentMethod.bitcoin,
              selectedValue: paymentSelected,

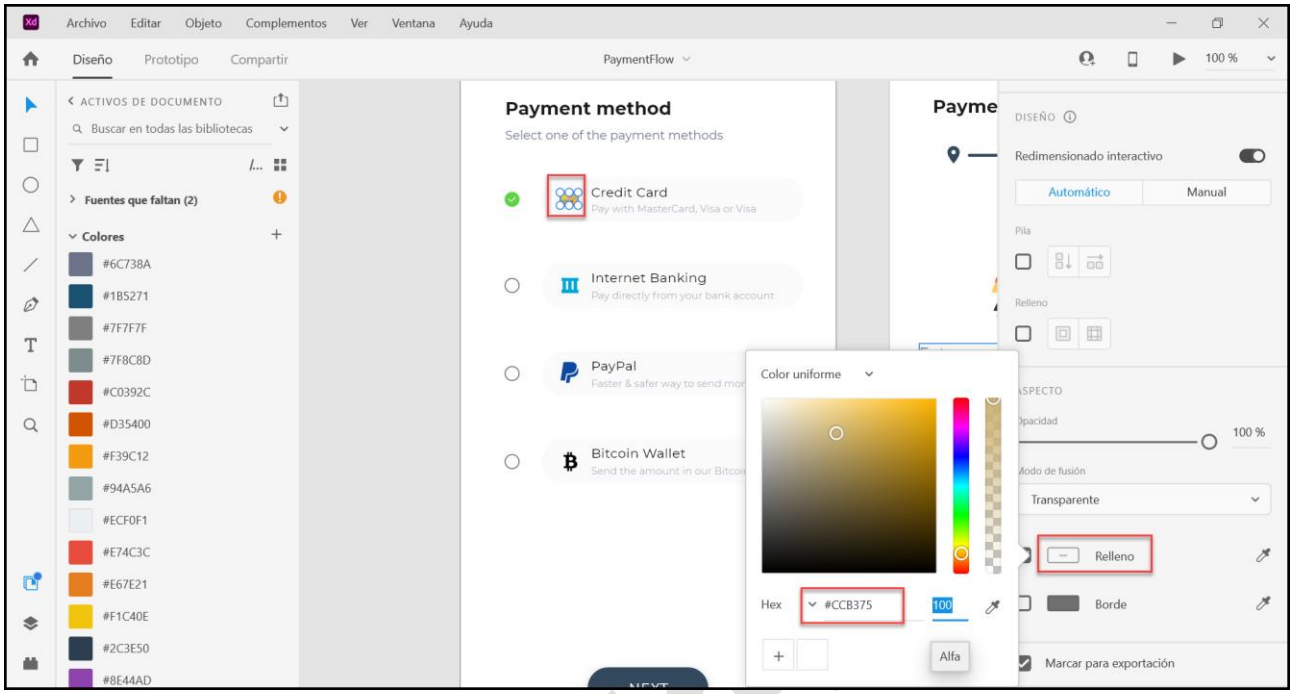
              paymentCallBack: _onPaymentChange,

            ), // PaymentMethodWidget
          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }

  void _onPaymentChange(PaymentMethod value) {
    setState(() {});
    paymentSelected = value;
  }

}
```

48. Abra el archivo **PaymentFlow** en **Adobe XD**. Con la ayuda de esta aplicación **investigue** el **color** para cada **icono** de las **formas de pago**.



49. Luego de extraer los valores de cada **icono** de la UI de las **formas de pago**, los colores serían:

ICONO	**COLOR**
Credit Card	#CCB375
Internet Banking	#00A7D9
PayPal	#0D47A1
Bitcoin Wallet	#000000

50. Con los valores **hexadecimales** obtenidos, en el proyecto **establezca** en la propiedad **color** del método **PaymentMethodWidget** los **colores** para el **icono** de cada método de pago. Por lo tanto, abra el archivo **payment_method_screen.dart** en la clase **_PaymentMethodState**, método **build**, **cambie** cada llamada al método **PaymentMethodWidget**. El código debería quedar como se muestra a continuación.

payment_method_screen.dart M

```dart
            ), // Text
            const Text(
              'Seleccione uno de los métodos de pago',
              style: TextStyle(
                fontSize: 12,
                color: Color(0xff6C738A),
              ), // TextStyle
            ), // Text
            const SizedBox(
              height: 20,
            ), // SizedBox
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.creditCard,

                color: Color(0xffCCB375),

              ), // Icon
              title: 'Tarjeta de crédito',
              subTitle: 'Pague con Mastercard o Visa',
              value: PaymentMethod.creditCard,
              selectedValue: paymentSelected,
              paymentCallBack: _onPaymentChange,
            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.warehouse,

                color: Color(0xff00A7D9),

              ), // Icon
              title: 'Transferencia bancaria',
              subTitle: 'Pague directo desde su cuenta',
              value: PaymentMethod.bank,
              selectedValue: paymentSelected,
              paymentCallBack: _onPaymentChange,
            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.paypal,

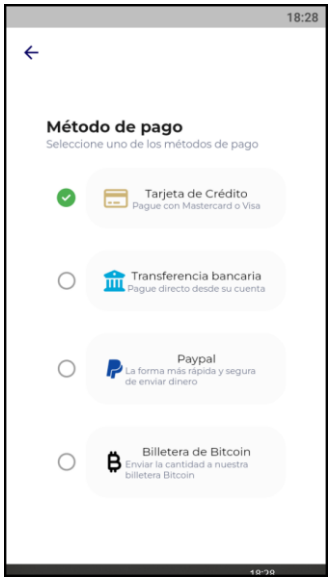
                color: Color(0xff0D47A1),

              ), // Icon
              title: 'Paypal',
              subTitle: 'La forma más rápida y segura de enviar diner
              value: PaymentMethod.paypal,
              selectedValue: paymentSelected,
              paymentCallBack: _onPaymentChange,
            ), // PaymentMethodWidget
            PaymentMethodWidget(
              icon: const Icon(
                FontAwesomeIcons.bitcoin,

                color: Color(0xff000000),
                      You, 1 second ago • Uncommitted changes
              ), // Icon
              title: 'Billetera de bitcoin',
              subTitle: 'Enviar la cantidad a nuestra billetera Bitcc
              value: PaymentMethod.bitcoin,
              selectedValue: paymentSelected,
              paymentCallBack: _onPaymentChange,
            ), // PaymentMethodWidget
          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }

  void _onPaymentChange(PaymentMethod value) {
    setState(() {});
    paymentSelected = value;
  }
}
```

51. **Ejecute** y **pruebe** la aplicación.

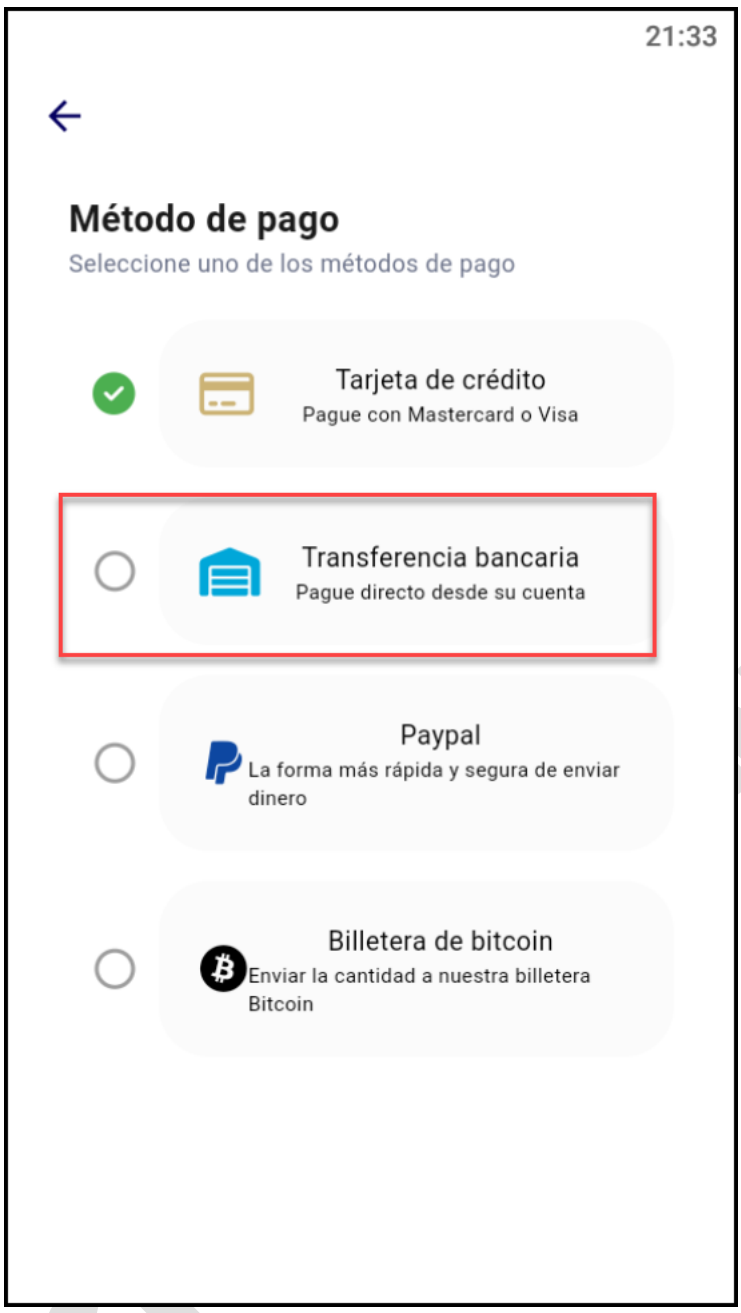


52. Puesto que el objetivo es tener una **interfaz lo más similar posible al mockup**, vamos a modificar el **padding** en la propiedad **body** del **Scaffold** de la clase **_PaymentMethodState**, para disminuir la distancia entre el **texto** "**Método de pago**" y las formas de pago. **Realice** el siguiente cambio en el **padding**.

payment_method_screen.dart

```
11        return PaymentMethodState();
12      }
13    }
14
15    class PaymentMethodState extends State<PaymentMethodScreen> {
16      PaymentMethod paymentSelected = PaymentMethod.crediCard;
17
18      @override
19      Widget build(BuildContext context) {
20        return Scaffold(
21          appBar: AppBar(
22            leading: IconButton(
23              icon: const Icon(Icons.arrow_back),
24              onPressed: () {},
25            ), // IconButton
26          ), // AppBar
27          body: Padding(
28
29            // padding: const EdgeInsets.all(45.0),
30            padding: const EdgeInsets.only(left: 30, right: 30, top: 10),
31
32            child: Column(
33              crossAxisAlignment: CrossAxisAlignment.start,
34              children: <Widget>[
35                const Text(
36                  'Método de pago',
37                  style: TextStyle(
38                    fontSize: 18,
39                    fontWeight: FontWeight.bold,
40                    color: Color(0XFF1A1A1A),
41                  ), // TextStyle
42                ), // Text
```

53. **Ejecute** y **pruebe** el proyecto.



54. Si el icono para la **transferencia bancaria** no es el mismo o parecido al que se muestra en el **mockup**, tome en cuenta que estamos utilizando la versión **10.6.0** del paquete **font_awesome_flutter**, esto lo podemos visualizar en el archivo **pubspec.yaml**. Por lo tanto, **cambie** en el archivo **payment_method_screen.dart**, en la clase **_PaymentMethodState**, en el método **build**, en la llamada del

PaymentMethodWidget para la **transferencia bancaria**, cambie el valor del **icono** para actualizar el icono a la nueva versión.

Código anterior:

payment_method_screen.dart M | pubspec.yaml

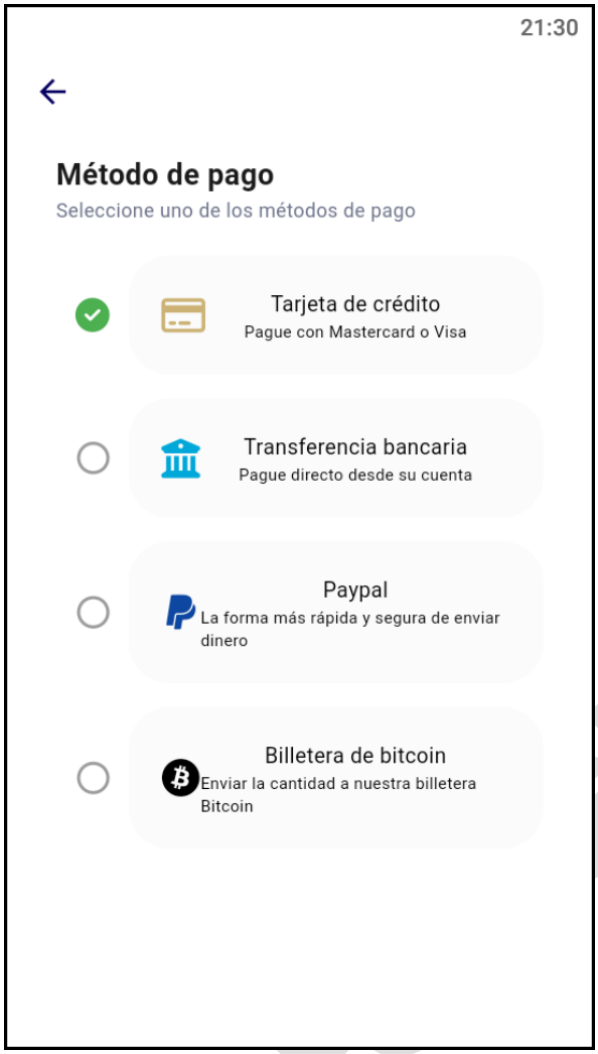
```
48                icon: const Icon(
49                  FontAwesomeIcons.creditCard,
50                  color: Color(0xffCCB375),
51                ), // Icon
52                title: 'Tarjeta de crédito',
53                subTitle: 'Pague con Mastercard o Visa',
54                value: PaymentMethod.creditCard,
55                selectedValue: paymentSelected,
56                paymentCallBack: _onPaymentChange,
57              ), // PaymentMethodWidget
58              PaymentMethodWidget(      You, 6 hours ago • Terminado punto 25
59                icon: const Icon(
60
61                  FontAwesomeIcons.warehouse,
62
63                  color: Color(0xff00A7D9),
64                ), // Icon
65                title: 'Transferencia bancaria',
66                subTitle: 'Pague directo desde su cuenta',
67                value: PaymentMethod.bank,
68                selectedValue: paymentSelected,
69                paymentCallBack: _onPaymentChange,
70              ), // PaymentMethodWidget
71              PaymentMethodWidget(
72                icon: const Icon(
73                  FontAwesomeIcons.paypal,
```

Código nuevo:

payment_method_screen.dart M | pubspec.yaml

```
48                icon: const Icon(
49                  FontAwesomeIcons.creditCard,
50                  color: Color(0xffCCB375),
51                ), // Icon
52                title: 'Tarjeta de crédito',
53                subTitle: 'Pague con Mastercard o Visa',
54                value: PaymentMethod.creditCard,
55                selectedValue: paymentSelected,
56                paymentCallBack: _onPaymentChange,
57              ), // PaymentMethodWidget
58              PaymentMethodWidget(
59                icon: const Icon(      You, 6 hours ago • Terminado punto 25
60
61                  FontAwesomeIcons.buildingColumns,
62
63                  color: Color(0xff00A7D9),
64                ), // Icon
65                title: 'Transferencia bancaria',
66                subTitle: 'Pague directo desde su cuenta',
67                value: PaymentMethod.bank,
68                selectedValue: paymentSelected,
69                paymentCallBack: _onPaymentChange,
70              ), // PaymentMethodWidget
```

55. **Ejecute** y **pruebe** el proyecto. Ahora la UI de la pantalla debería tener una apariencia similar a la que se muestra a continuación.



Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

Working with Callback In Flutter
https://medium.flutterdevs.com/working-with-callback-in-flutter-89dc207cba37
Filas y Columnas en Flutter
https://blog.pleets.org/article/es/filas-y-columnas-en-flutter